

P2 - Saimazoom

Objetivo

En esta práctica el objetivo es implementar un sistema distribuido sencillo usando RabbitMQ (colas de mensajes). Hay que implementar toda la cadena logística de un marketplace llamado Saimazoom. En este escenario existen los siguientes actores:

- **Cliente**, que realiza y gestiona pedidos de productos.
- **Controlador** central, que gestiona todo el proceso.
- **Robots**, que se encargan de buscar los productos en el almacén y colocarlos en las cintas transportadoras.
- **Repartidores**, encargados de transportar el producto a la casa del cliente.

Para ello, será necesario implementar las operaciones habituales en este tipo de escenarios, que se realizarán en el siguiente orden: creación/consulta/cancelación de un pedido. La cancelación del pedido, solo se permite antes que el pedido haya empezado a ser tramitado por un repartidor. Se entrega un documento de requisitos, a completar, para definir de forma más clara las funcionalidades necesarias.

Para poder llevar a cabo la práctica, deberás comenzar realizando una serie de decisiones de diseño (documentándolas adecuadamente a través de un diagrama de clases, diagrama de estados) que incluirán, como mínimo, las siguientes:

- Diagrama de estados en los que puede estar un pedido en cada momento
- Diagrama de clases
- Formato y sintaxis de los mensajes (RFC)

Consejos de desarrollo

- Hay que entender bien qué es necesario implementar y qué no. Por ejemplo, no hace falta llevar un registro de los productos que hay en el almacén, podemos simular que un robot no encuentra un producto con una probabilidad.
- Para facilitar el desarrollo, puede utilizarse el siguiente truco: crear los distintos actores, en lugar de como programas independientes, como hilos dentro del proceso principal (controlador). En cualquier caso, antes de entregar la práctica debe revertirse el cambio.
- Siempre es buena idea atacar el desarrollo dividiendo el problema en partes, para aislar y reducir las posibles fuentes de errores. Así, para desarrollar y probar el repartidor, por ejemplo, se pueden generar e insertar en la cola mensajes de prueba, de forma que no haga falta que esté ya implementado ningún otro módulo.

Calidad del código

- El código fuente debe estar correctamente comentado
- Se valorará que el sistema se organice como un paquete de Python

- Se deben implementar un número suficiente de tests unitarios.
Idealmente, se recomienda utilizar una metodología de desarrollo TDD (Test-Driven Development)

1. Funcionalidad

1.1. Controlador

El controlador es el principal actor del sistema y se encarga de gestionar los pedidos y la comunicación con el resto de actores. Son necesarias varias colas para dicha comunicación, como por ejemplo la cola para el trabajo de robots o de los repartidores, pero seguramente hagan falta más. Es necesario diseñar el controlador definiendo qué colas son necesarias y cómo será la interacción con el resto de actores del sistema.

Respecto a los pedidos, el controlador tiene que tener un registro de qué pedidos han sido realizados y llevar un control de su estado. También hay que tener un registro de los clientes y de qué pedidos ha realizado cada uno. Si un cliente no registrado intenta hacer un pedido, el sistema debe contestar con un mensaje indicando que no está registrado. Sin embargo, NO hace falta modelar los productos, no hay que tener un registro de productos válidos.

Por último, es necesario que el sistema persista su estado en todo momento, para lo que pueden utilizarse un par de opciones: i) estado en ficheros (utilizando la librería Pickle, o formato CSV) o, ii) usando una base de datos al lanzar el controlador; en este caso también hay que guardar los datos antes de terminar la ejecución.

Actividad previa

[Examen - Práctica 1 \(2313-2393\)](#)

Ir a...

Siguiente actividad

[Recursos P2](#)

[Política de privacidad](#) [Contacto](#) [Aviso legal](#) [Accesibilidad](#) [UAM Covid 19](#)